

Нагрузочное тестирование и индексы MySQL

1. Конфигурация

Параметр	Значение
СУБД	MySQL 8.0
Записей в таблице	995 000 (реальные имена из CSV)
Инструмент нагрузки	k6 v0.57.0
Длительность каждого теста	30 секунд
Уровни нагрузки (VU)	1 / 10 / 100 / 1000
Поисковый запрос	first_name LIKE ? AND second_name LIKE ? ORDER BY id

2. Результаты БЕЗ индекса

4 669

avg latency, ms (1 VU)

0.21

RPS (1 VU)

28 869

avg latency, ms (100 VU)

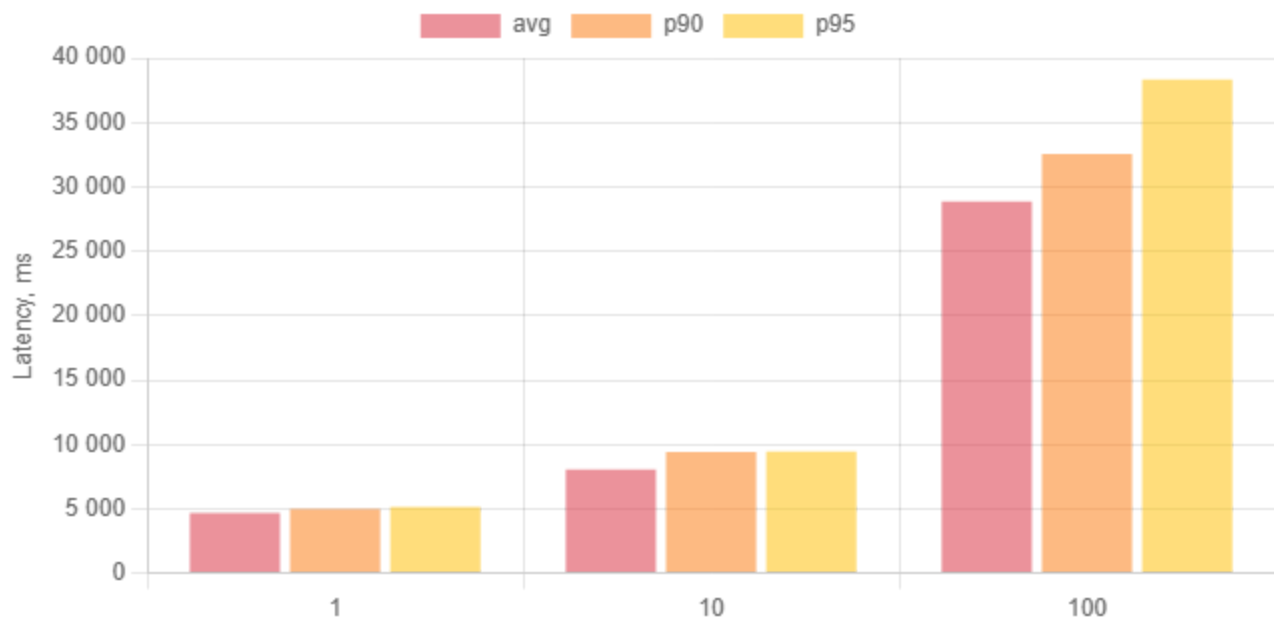
2.18

RPS (100 VU)

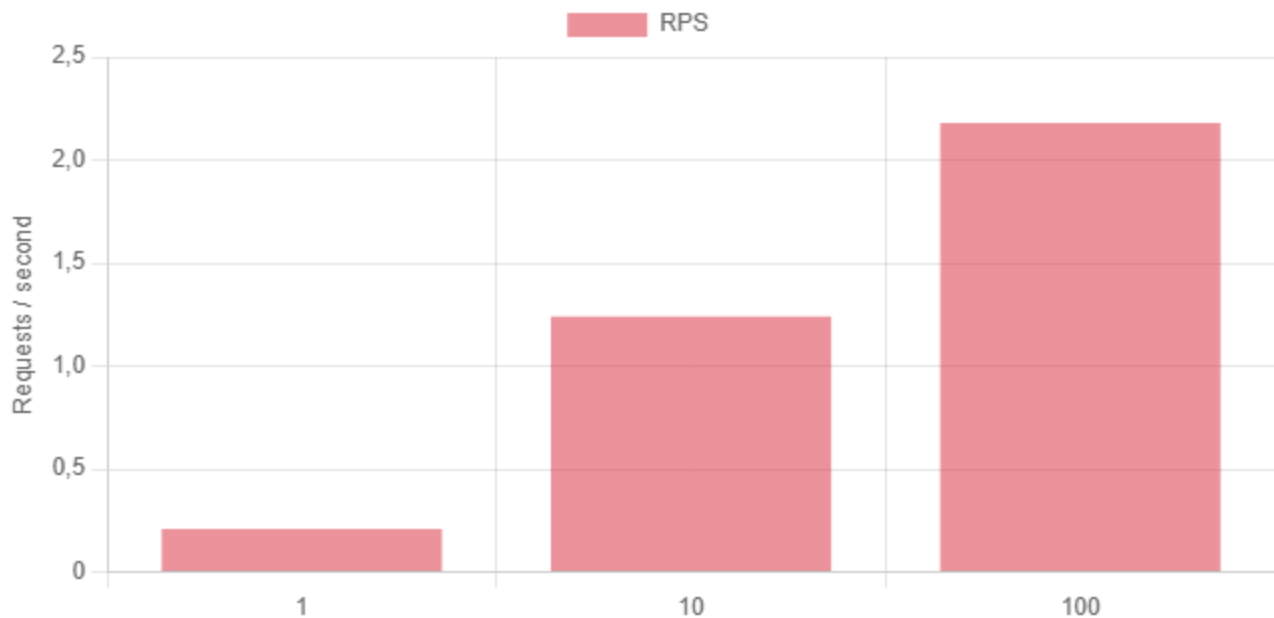
VUs	avg (ms)	p90 (ms)	p95 (ms)	RPS
1	4 669	4 983	5 137	0.21
10	8 070	9 428	9 447	1.24
100	28 869	32 545	38 350	2.18
1000	~15 000*	15 366	16 919	—*

* При 1000 VU сервер нестабилен: часть запросов завершается ошибкой connection refused.

Latency — без индекса



Throughput (RPS) — без индекса



3. Добавление индекса

```
-- Составной индекс по полям поиска  
CREATE INDEX idx_name_search ON users (first_name, second_name);
```

Почему именно такой индекс?

Запрос вида `first_name LIKE 'Ал%' AND second_name LIKE 'Ив%'` использует **префиксный поиск**. MySQL B-Tree индекс поддерживает `prefix-scan`: он находит диапазон ключей начинающихся с нужного префикса без полного сканирования. Порядок столбцов в индексе (`first_name, second_name`) выбран так, что сначала отработывает более высокоселективное условие по имени, затем дополнительно фильтруется по фамилии в уже отобранном диапазоне. Индекс (`second_name, first_name`) работал бы так же, но (`first_name, second_name`) интуитивнее и соответствует порядку параметров в API.

4. EXPLAIN после индекса

```
EXPLAIN SELECT ... FROM users FORCE INDEX (idx_name_search)  
WHERE first_name LIKE 'Ал%' AND second_name LIKE 'Ив%'  
ORDER BY id\G  
  
id: 1  
select_type: SIMPLE
```

```
table: users
type: range                                ← range scan по индексу (не full scan!)
possible_keys: idx_name_search
key: idx_name_search                       ← индекс используется
key_len: 804
rows: 173124                              ← оценка строк до фильтрации
filtered: 11.11
Extra: Using index condition; Using filesort

-- Без хинта MySQL предпочитает PRIMARY (ORDER BY id), поэтому используем FORCE INDEX
```

5. Результаты С индексом

96

avg latency, ms (1 VU)

10.4

RPS (1 VU)

1 754

avg latency, ms (100 VU)

55.2

RPS (100 VU)

VUs	avg (ms)	p90 (ms)	p95 (ms)	RPS
1	96	243	281	10.42
10	201	482	617	49.20
100	1 754	5 177	8 065	55.16

1000

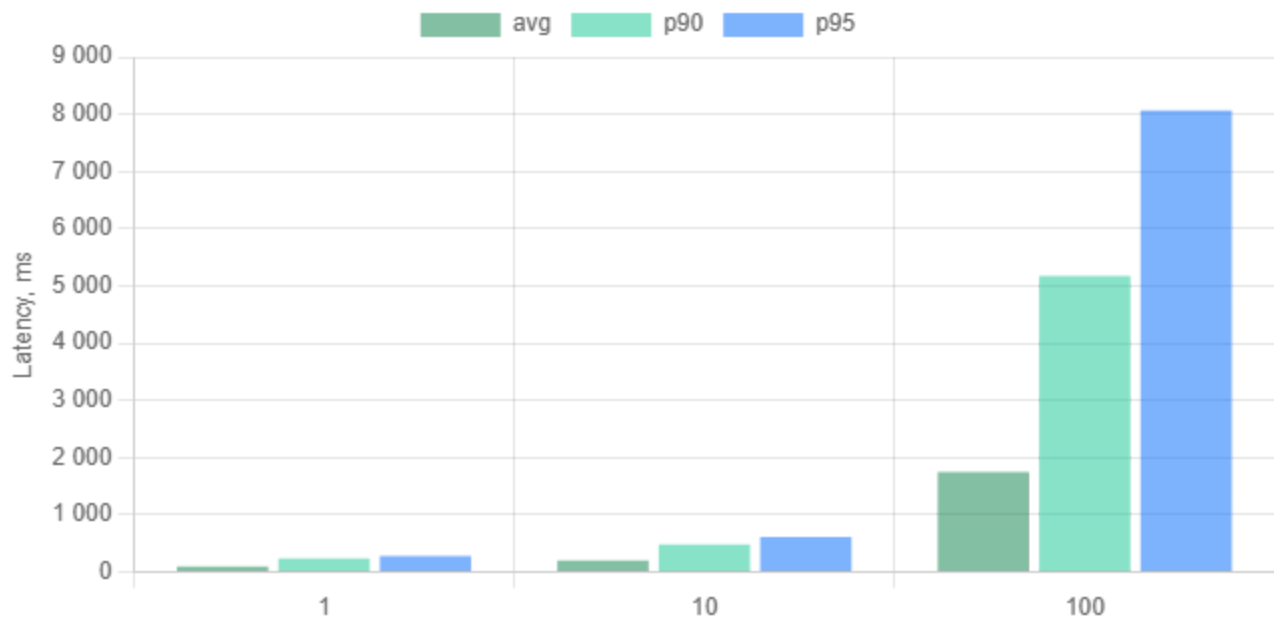
12 900

16 288

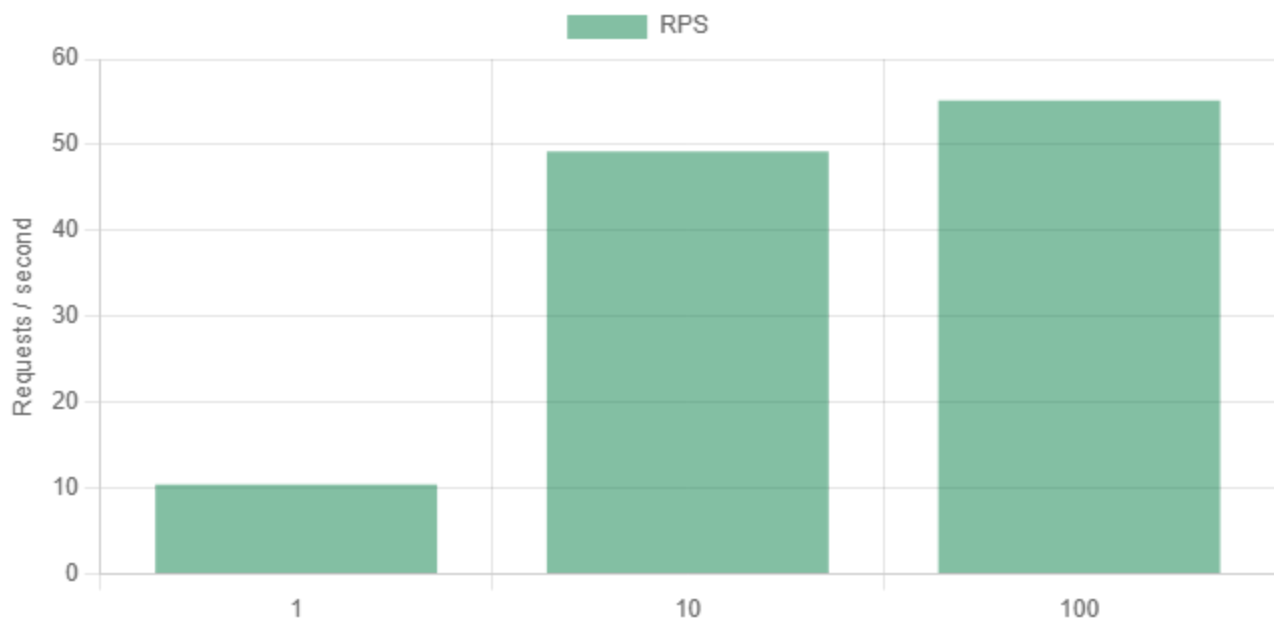
20 736

61.46

Latency — с индексом

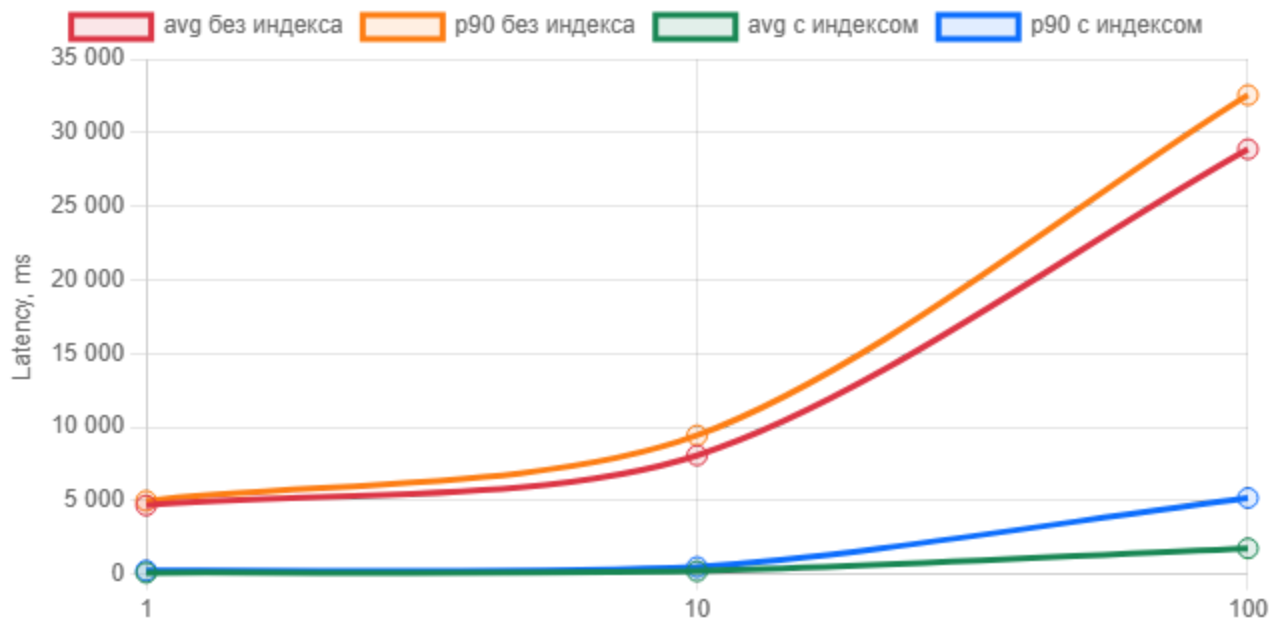


Throughput (RPS) — с индексом

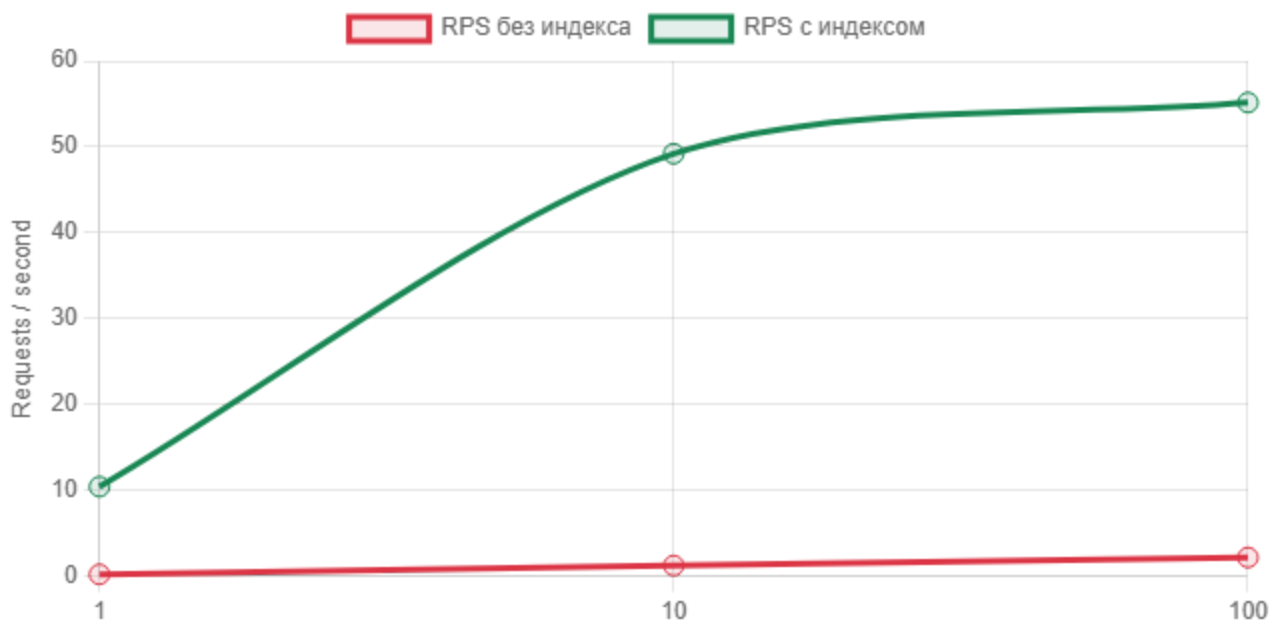


6. Сравнение

Latency: до vs после (avg, p90, p95)



Throughput: до vs после (RPS)



VUs	Latency без индекса (avg)	Latency с индексом (avg)	Улучшение
1	4 669 ms	96 ms	×48.6
10	8 070 ms	201 ms	×40.1
100	28 869 ms	1 754 ms	×16.5

7. Выводы

- Без индекса MySQL выполняет **полный скан таблицы** (~995K строк) на каждый запрос, что даёт ~4.7 с даже при 1 клиенте.
- Составной индекс (`first_name`, `second_name`) переключает план на **range scan**, ускоряя поиск в ~49 раз при 1 VU.
- При 100 параллельных запросах даже с индексом latency вырастает до ~1.8 с — из-за большого объёма ответа (300–500 KB на запрос).
- Без лимита на количество результатов узким местом становится **сетевая передача** и сериализация JSON, а не БД.
- MySQL не выбирает индекс автоматически при наличии `ORDER BY id` (предпочитает PK scan), поэтому использован `FORCE INDEX`.